

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



[@Andriessens_C](https://twitter.com/Andriessens_C)

┌ GRUNDLEGENDE PRINZIPIEN IM SOFTWARE ENGINEERING

Die braucht man immer wieder...
deshalb verstehen und gut einprägen



Themen in dieser Vorlesung



Einführung

Softwareprozessmodelle

Anforderungen

Architektur und Entwurf

Implementierung

Softwarequalität und Sicherheitsfragen

Lernziele

- Nach welchen **Grundsätzen** kann man vorgehen,
 - um mit **Komplexität** und **Veränderung** erfolgreich umzugehen, uns
 - Werkzeuge und Methoden zum Umgang mit Komplexität und Veränderung zu konstruieren und so
 - erfolgreich in Softwareprojekten zu arbeiten und darin
 - erfolgreich große Softwaresysteme zu gestalten?
- **Grundlegende Prinzipien im Software Engineering kennenlernen und verstehen**

Software Engineering – Grundlagen



Software Engineering – Grundlagen

Prinzipien sind Grundsätze, die man seinem Handeln zugrundelegt.

Genauer: Ein allgemeingültiger Grundsatz, der aus der Verallgemeinerung von Gesetzen und wesentlichen Eigenschaften der objektiven Realität abgeleitet ist und als Leitfaden dient.



Software Engineering – Grundlagen

Methoden sind planmäßig angewandte, begründete Vorgehensweisen bzw. Handlungsvorschriften zur Erreichung von festgelegten Zielen (i.a. im Rahmen festgelegter Prinzipien).

Die *Handlungsvorschrift* beschreibt, wie, ausgehend von gegebenen Bedingungen, ein Ziel mit einer festgelegten Schrittfolge erreicht wird.

Methoden sollen anwendungsneutral sein, dieselben Methoden sollen daher z.B. für verschiedene Programmier-sprachen gelten.



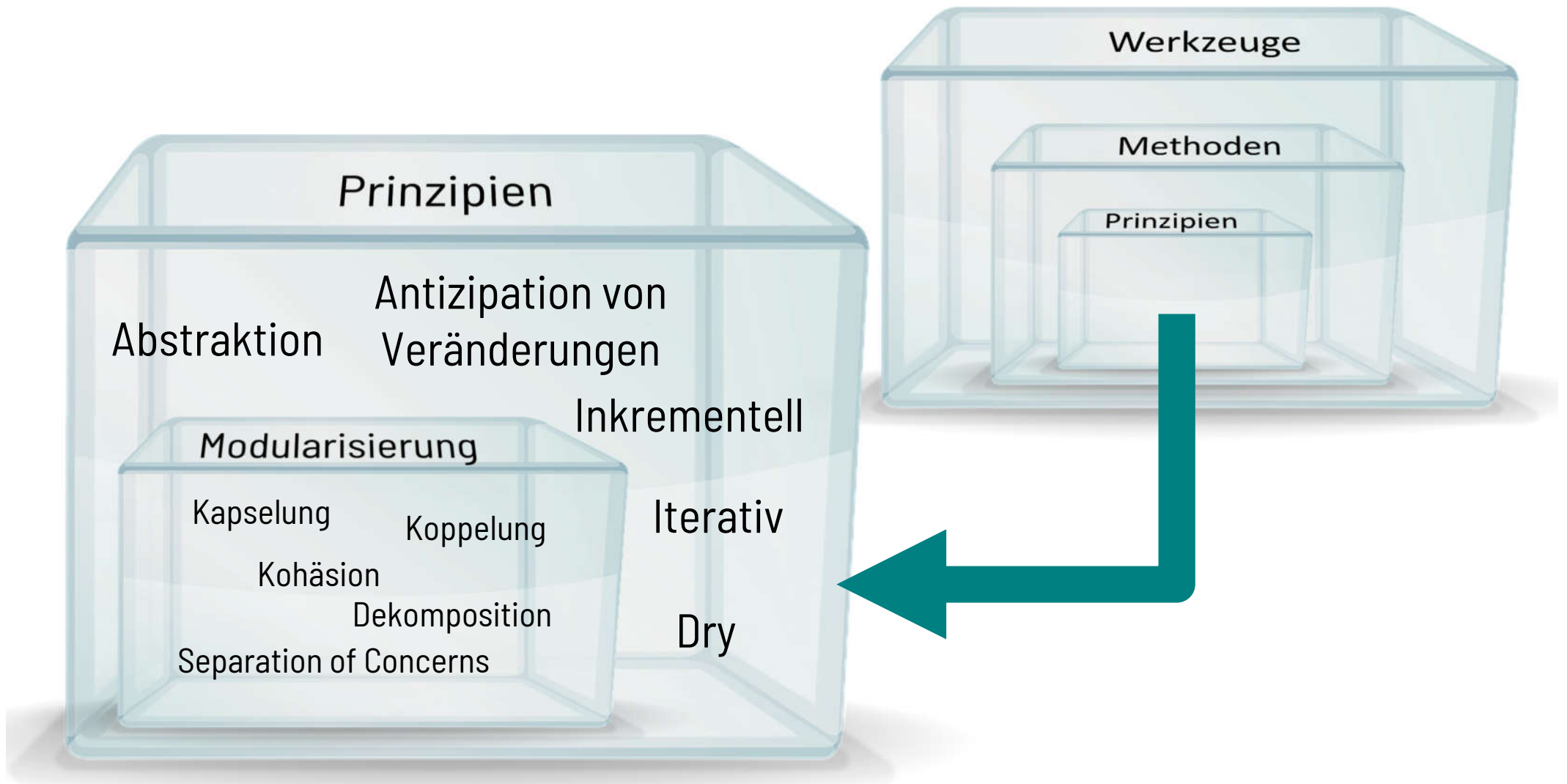
Software Engineering – Grundlagen

Werkzeuge dienen der automatisierten Unterstützung von Methoden und Verfahren.

Werkzeuge im Software Engineering sind programmtechnische Mittel zum automatisierten Bearbeiten von „Informationsmengen“.



Software Engineering - Prinzipien



Abstraktion

Software Engineering-Prinzipien

Abstraktion: Schaffung einer vereinfachten Sicht auf Konzepte oder Objekte.

- Einnahme eines Standpunktes und Entfernung davon unnötiger Eigenschaften. Wie?
 - Zusammenfassung (Aggregation, Klassifikation)
 - Auswahl (Selektion)
- Konzentration auf das Wesentliche – bei Bedarf Konkretisierung

Abstraktion

Software Engineering-Prinzipien

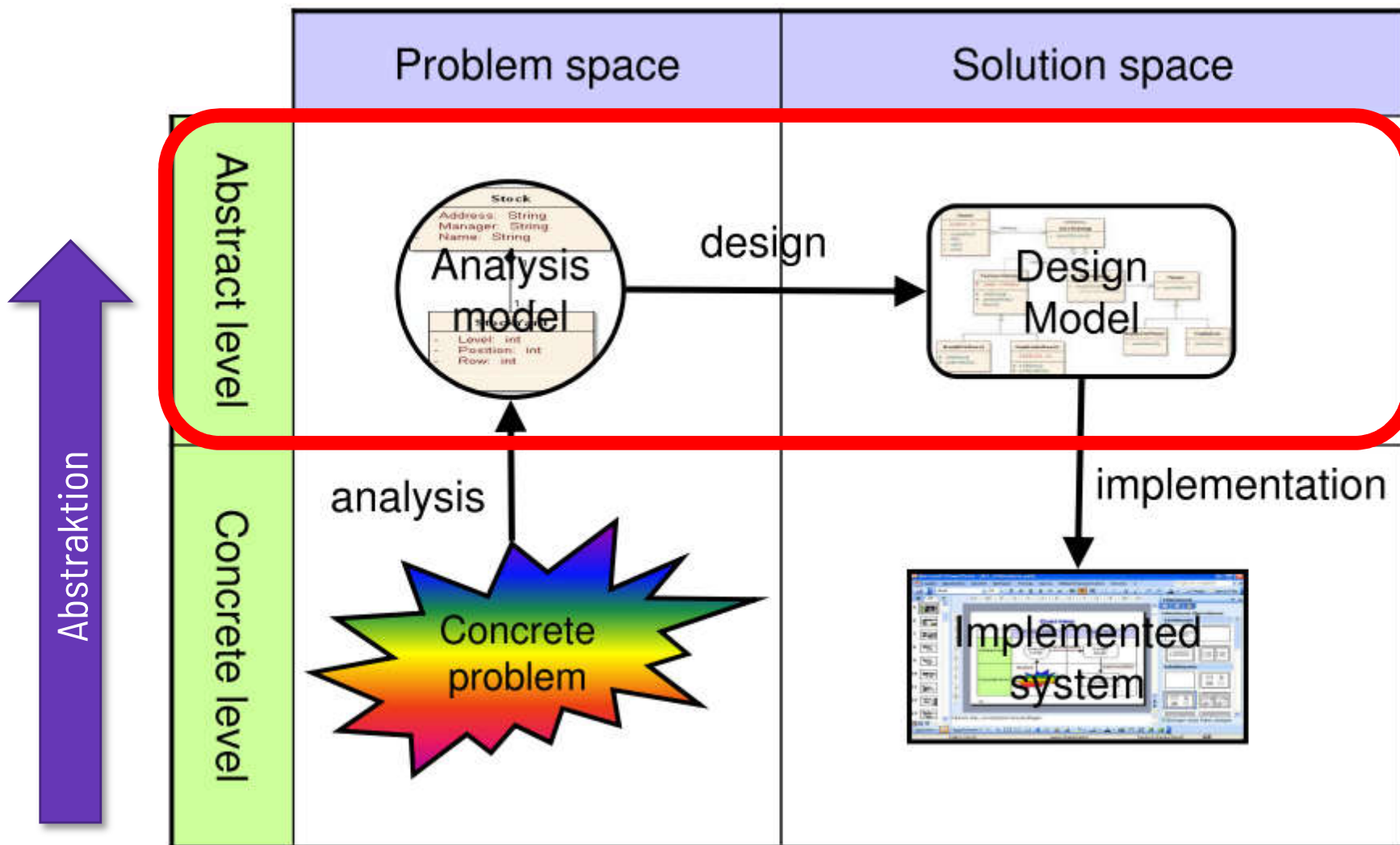
Beispiele:

- Modelle als Abstraktionen der Realität
- Beispiel: Anforderungen $\Rightarrow$ Code
- 1 Spezifikation $\Rightarrow$ n Realisierungen
- Fokussierung auf das Wesentliche in Analysephase für ein Hotelbuchungssystem:

Unterscheidung zwischen Einzel- und Doppelzimmer, Rechnungslayout werden zunächst ignoriert. Später: Preisunterschied zwischen Einzel- und Doppelzimmer

Abstraktion

Software Engineering-Prinzipien



Modularisierung

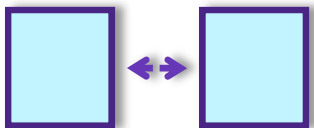
Software Engineering-Prinzipien

Modularisierung ist die Zerlegung eines komplexen Sachverhaltes in kleinere, handhabbare Einheiten.

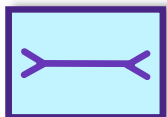
Natürliche und sehr wirksame Reaktion des Menschen im Umgang mit Komplexität. Wird entlang der „3K“ umgesetzt:



Kapselung: Trennung Schnittstelle / versteckte Implementierung



Lose Koppelung zwischen den Einheiten

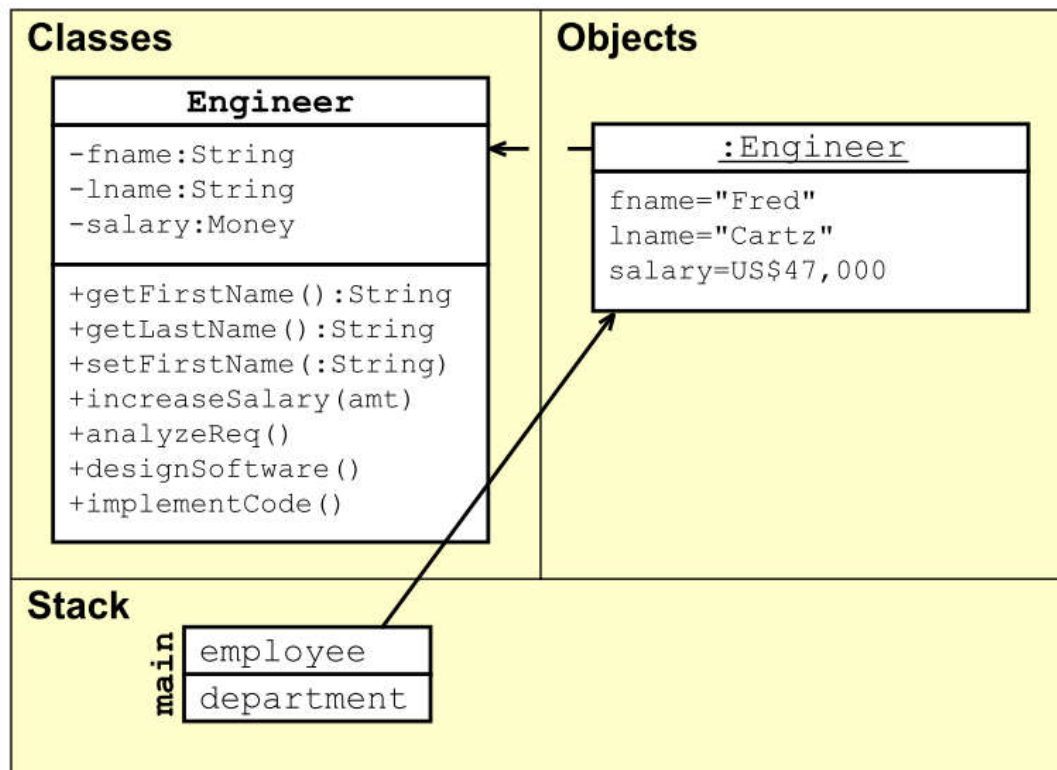


Hohe Kohäsion in den Einheiten

Modularisierung: Kapselung

Software Engineering-Prinzipien

Kapselung: Die Trennung der Schnittstelle von der versteckten Implementierung.



Implementierung (Details des „WIE“) wird versteckt, nur das für Außen nötigste wird über eine Schnittstelle angeboten. Dadurch wird das „WIE“, die Implementierung unabhängig änderbar. Im Beispiel nutzt

✗ keine Kapselung, nur

✓ nutzt Kapselung

✗ `name = employee.fname;`

✗ `employee.fname = "Samantha";`

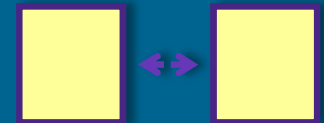
✓ `name = employee.getFirstName();`

✓ `employee.setFirstName("Samantha");`

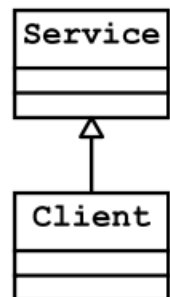
Modularisierung: Koppelung

Software Engineering-Prinzipien

Koppelung ist ein Maß der Abhängigkeit zwischen zwei Einheiten eines Systems.



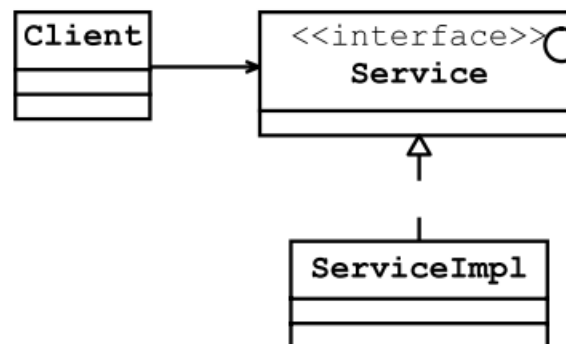
Tight Coupling



Looser Coupling



Looser Abstract Coupling



No Coupling



Modularisierung: Koppelung

Software Engineering-Prinzipien

Arten von Koppelung

- Kopplung durch Aufruf
 - Kopplung durch Erzeugung
 - Kopplung durch Datenstrukturen
 - Kopplung durch Hardware oder Laufzeitumgebungen
 - Kopplung über die Zeit (Zustand)
- **Kopplung existiert auf allen Ebenen der Struktur eines Systems**

Modularisierung: Koppelung

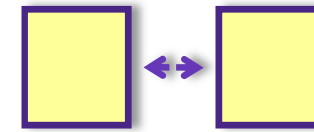
Software Engineering-Prinzipien

Kopplung bindet Systembestandteile aneinander

- Je enger die Koppelung, desto wahrscheinlicher ist Notwendigkeit gemeinsamer Änderung -> Aufwand
- Verschiedene Ansätze für lose Koppelung. Etwa:
 - Systemstruktur: SOA
 - Klassen: Command Pattern, Reflection, Nachrichten (JMS)

Koppelung: Beispiel (1/4)

Software Engineering-Prinzipien



```
public class Traveller {  
    Car c = new Car();  
  
    public void startJourney() {  
        c.move();  
    }  
}
```

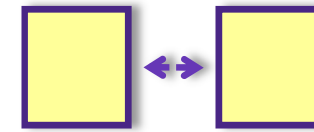
- Was passiert, wenn wir das **Auto gegen den Zug austauschen** wollen?
- Wie eng sind Traveller und die Verkehrsmittel aneinander gebunden?
- Wie leicht können wir **neue Verkehrsmittel** hinzufügen?

```
public class Car {  
  
    public void move() {  
        // moves car  
    }  
}
```

```
public class Train {  
  
    public void move() {  
        // moves train  
    }  
}
```

Koppelung: Beispiel (2/4)

Software Engineering-Prinzipien



```
public class Traveller {  
    Car c = new Car();  
  
    public void startJourney() {  
        c.move();  
    }  
}
```

Entkoppelung über Interface, Schritt 1

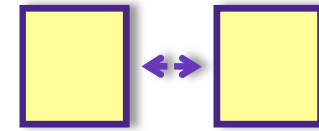
```
public interface MeansOfTransport  
{  
    void move();  
}
```

```
public class Car {  
  
    public void move() {  
        // moves car  
    }  
}
```

```
public class Train {  
  
    public void move() {  
        // moves train  
    }  
}
```

Koppelung: Beispiel (3/4)

Software Engineering-Prinzipien



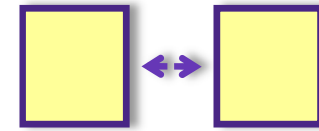
Entkoppelung über Interface, Schritt 2

```
public class Traveller {  
    MeansOfTransport c;  
  
    public void setMeansOfTransport(MeansOfTransport mot) {  
        this.c = mot;  
    }  
  
    public void startJourney() {  
        c.move();  
    }  
}
```

```
public interface MeansOfTransport {  
    void move();  
}
```

Koppelung: Beispiel (4/4)

Software Engineering-Prinzipien



Entkoppelung über Interface, Schritt 3

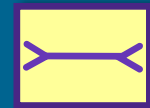
```
public class Car implements MeansOfTransport {  
  
    public void move() {  
        // moves car  
    }  
}
```

```
public class Train implements MeansOfTransport {  
    public class Airplane implements MeansOfTransport {  
        public class Bus implements MeansOfTransport {  
            public class ... implements MeansOfTransport {  
  
                public void move() {  
                    // moves ...  
                }  
            }  
        }  
    }  
}
```

Modularisierung: Kohäsion

Software Engineering-Prinzipien

Zusammenhalt: Maß für die Fokussierung der Einheiten eines Systemteils auf ein Ziel



- **Niedrige Kohäsion:**
Viele Funktionen ohne Zusammenhang in großer Einheit („bunt gemischt“)
- **Hohe Kohäsion:**
Wenige und nur auf einander bezogene Funktionen in kleinerer Einheit („sortiert“)
- Konzept überträgt sich auf andere Einheiten wie Subsysteme

**Niedrige Kohäsion:
Schwierig zu warten**

SystemServices
+deleteDepartment() +deleteEmployee() +login() +logout() +makeDepartment() +makeEmployee() +retrieveDeptByID() +retrieveEmpByName()

**Hohe Kohäsion:
Leicht zu warten**

LoginService
+login() +logout()

EmployeeService
+deleteEmployee() +makeEmployee() +retrieveEmpByName()

DepartmentService
+deleteDepartment() +makeDepartment() +retrieveDeptByID()

Modularisierung: Separation of Concerns – SoC

Software Engineering-Prinzipien

- Fokussierung auf individuelle Aspekte und Belange beim Bau großer Systeme
 - Produktebene: Funktionen, GUI, Zuverlässigkeit, etc.
 - Prozessebene: Entwicklung, Zeitplan, Methodeneinsatz, Controlling, Management
- Zeit, Qualitätsattribute, Sichten, Komponenten, Verantwortlichkeiten
- Unterstützt Erhöhung der Kohäsion



Beispiel SoC: Formel 1 Auto

Software Engineering-Prinzipien

Belange und Verantwortlichkeiten

- Motor
 - Mercedes
- Chassis
 - McLaren
- Reifen
 - Bridgestone



Modularisierung: Top-Down vs. Bottom-Up

Software Engineering-Prinzipien

Bei der Zerlegung von großen Einheiten in gekapselte kleinere gibt es zwei Herangehensweisen:

- **Top-Down:** Herauslösen von Einzelteilen aus Gesamtbild
- **Bottom-Up:** Aufbau des Gesamtbildes aus Einzelteilen (auch "Komposition")



Modularisierung: Zusammenfassung

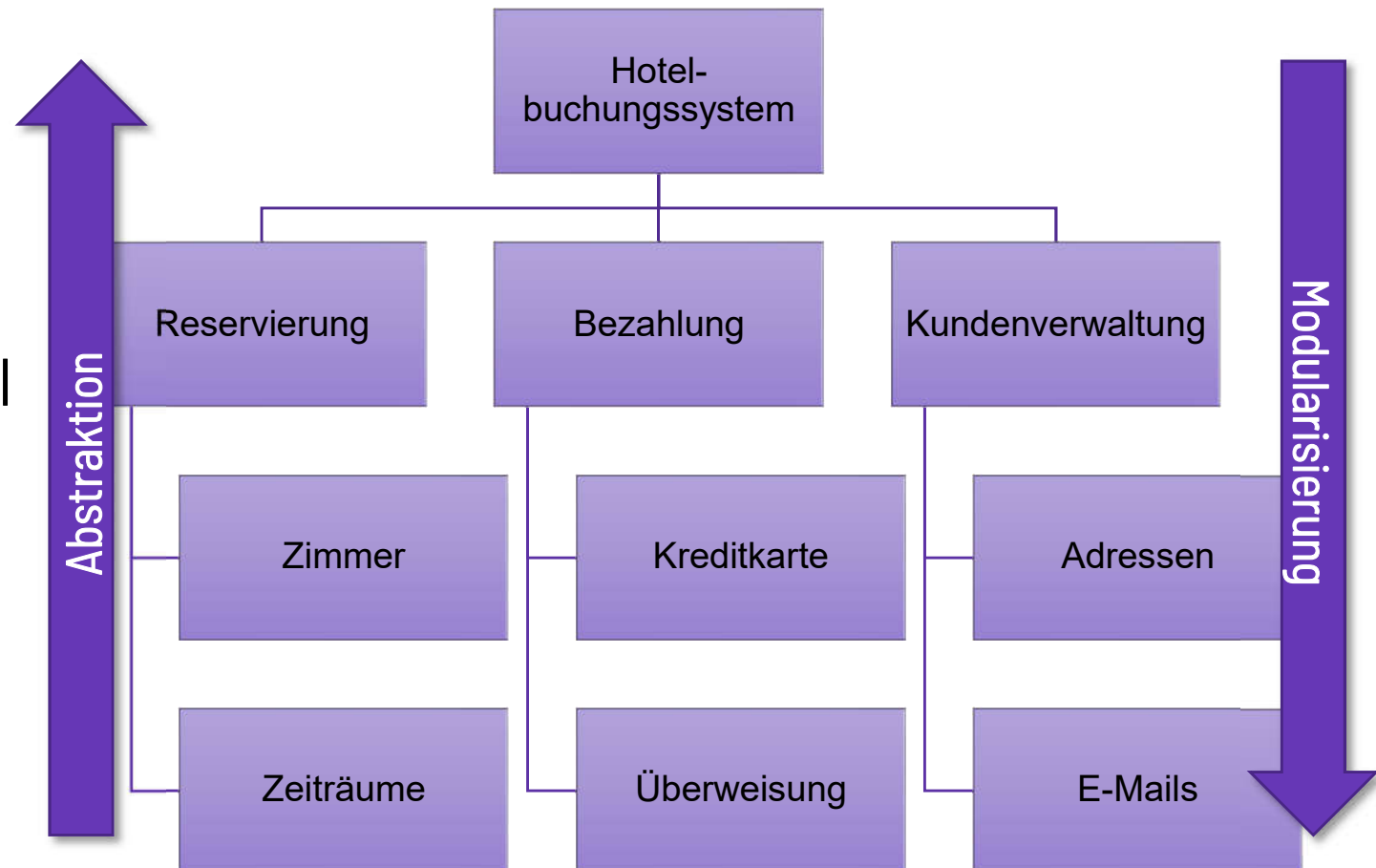
Software Engineering-Prinzipien

- Aufteilung eines komplexen Sachverhaltes (Systems, Problems) in einzelne, leichter handhabbare Einzelteile (Bausteine)
 - Auch: Grad (Stärke) solcher Aufteilung
- Verringert Komplexität, erhöht Wiederbenutzung und Verständlichkeit
- Aufteilung erfolgt Top-Down oder Bottom-Up und heißt selbst **Dekomposition** (bzw. **Komposition**)
- Modularisierung sollte entlang „Separation of Concerns“ erfolgen
- **Ziel** der Aufteilung ist immer: **hohe Kohäsion** in gut **gekapselten** Systemteilen, die dann mit **loser Kopplung** verbunden sind



Zusammenhang von Abstraktion und Modularisierung Software Engineering-Prinzipien

- **Abstraktion:**
Verringert die Anzahl von Elementen
 - **Modularisierung:**
Erhöht die Anzahl von Elementen
- Beides kann vereinfachen – und auch erschweren!



DRY

Software Engineering-Prinzipien

Don't Repeat Yourself

- Löse ein Problem einmal,
vermeide Redundanzen
- Wiederverwendung
zur Reduzierung und Vereinfachung
- Bessere Verständlichkeit
- Bessere Wartbarkeit: Leichtere und schnellere Umsetzung von
Änderungen mit weniger Fehlern

DRY

Software Engineering-Prinzipien

Beispiele, die DRY verletzen:

- Wiederholung von Dateinamen an mehreren Stellen im Quelltext
- Wiederholung der gleichen Logik an mehreren Stellen im Quelltext
- Änderungen des Namens oder des Pfades $\Rightarrow$ Alle Stellen im Quelltext müssen geändert werden $\Rightarrow$ erheblicher Aufwand + Fehlerquelle

Beispiele zur Umsetzung von DRY:

- Einsatz von Konstanten (Java: `static final ...`)
- Abstrakte Basisklassen
- Wiederholte Logik in Methoden umwandeln

DRY: Beispiel

Software Engineering-Prinzipien

```
public class Line {  
    Point start;  
    Point end;  
    double length;  
}
```



```
public class Line {  
    Point start;  
    Point end;  
    double length() {  
        return start.distanceTo(end);  
    }  
}
```



Aufweichung von DRY durch Einsatz von Kapselung

Software Engineering-Prinzipien

- Länge berechnen kostet Zeit.
 - Viele Linien, häufige Berechnung $\Rightarrow$ viel Zeit
 - Aber DRY? Was tun?
 - Idee: Wir weichen DRY auf und speichern die letzte Berechnung aus Performancegründen.
 - Der Trick dabei ist: Dank Kapselung die Auswirkung begrenzen!
- Prinzipien kombinieren kann helfen!

Aufweichung von DRY durch Einsatz von Kapselung

Software Engineering-Prinzipien

```
public class Line {  
    private Point start;  
    private Point end;  
    private double length;  
  
    public Line(Point start, Point end) {  
        this.start = start;  
        this.end = end;  
        calculateLength();  
    }
```

Beachten Sie die Nutzung von Sichtbarkeiten und getter-/setter-Methoden! Die Klasse Line versteckt ihre Implementierung (und damit den Umgang mit length) und steuert explizit die Zugriffe auf die Implementierung (und start und end).

```
public void setStart(Point p) { this.start = p; calculateLength(); }  
public void setEnd(Point p)   { this.end   = p; calculateLength(); }  
  
public Point getStart()       { return start; }  
public Point getEnd()         { return end;   }  
  
public double getLength()     { return length; }  
  
private void calculateLength() { this.length = start.distanceTo(end); }  
};
```


Antizipation von Veränderungen

Software Engineering-Prinzipien

- Hohe Änderungsraten bei Realisierung großer Softwareprojekte
 - Fehlerbeseitigung
 - Evolution (neue und sich ändernde Anforderungen)
- Ziel: Identifikation wahrscheinlicher Änderungen und Planung für Veränderung
 - Anforderungen häufig unklar zum Beginn der Realisierung
 - Anwender und (geschäftliche) Umgebungen ändern sich ständig
- Basis für Wiederbenutzbarkeit
- Wichtig: auch für Software Prozesse(!) relevant



Inkrementelles / iteratives Vorgehen

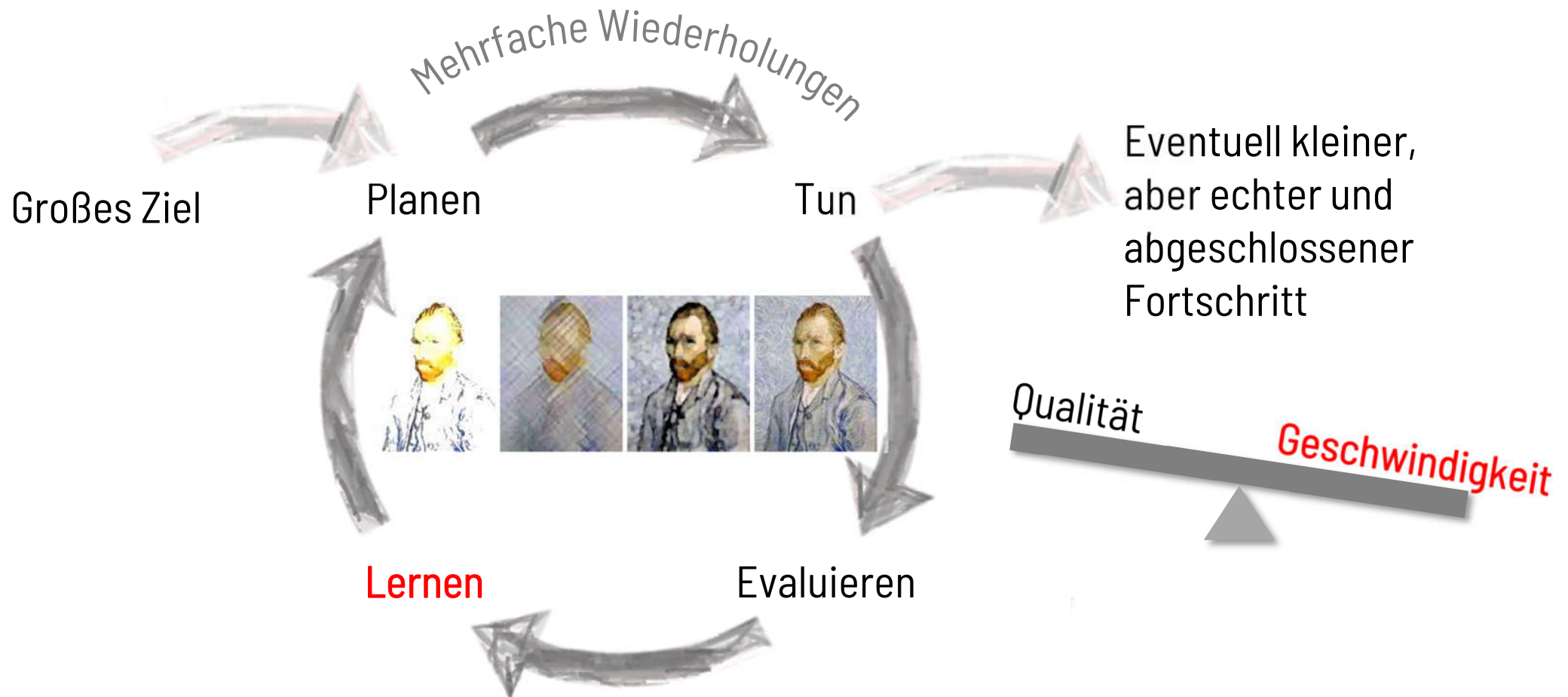
Software Engineering-Prinzipien

Schrittweises Vorgehen zur erfolgreichen Zielerreichung: **Inkrementell** = hinzufügend, **iterativ** = verfeinernd

- Frühes Feedback ist Basis für kontrollierte Evolution, sofern Anforderungen initial unvollständig oder unklar sind
- Inkrementeller Fokus auf verschiedene Qualitätsattribute wie beispielsweise Performanz
- Prototypen als Liefergegenstände
- Enge Verbindung zum Prinzip der Antizipation von Veränderungen
- Hohe Anforderungen an Management der Versionen („Konfigurationsmanagement“), Dokumentation, Inbetriebnahme („Deployment“)

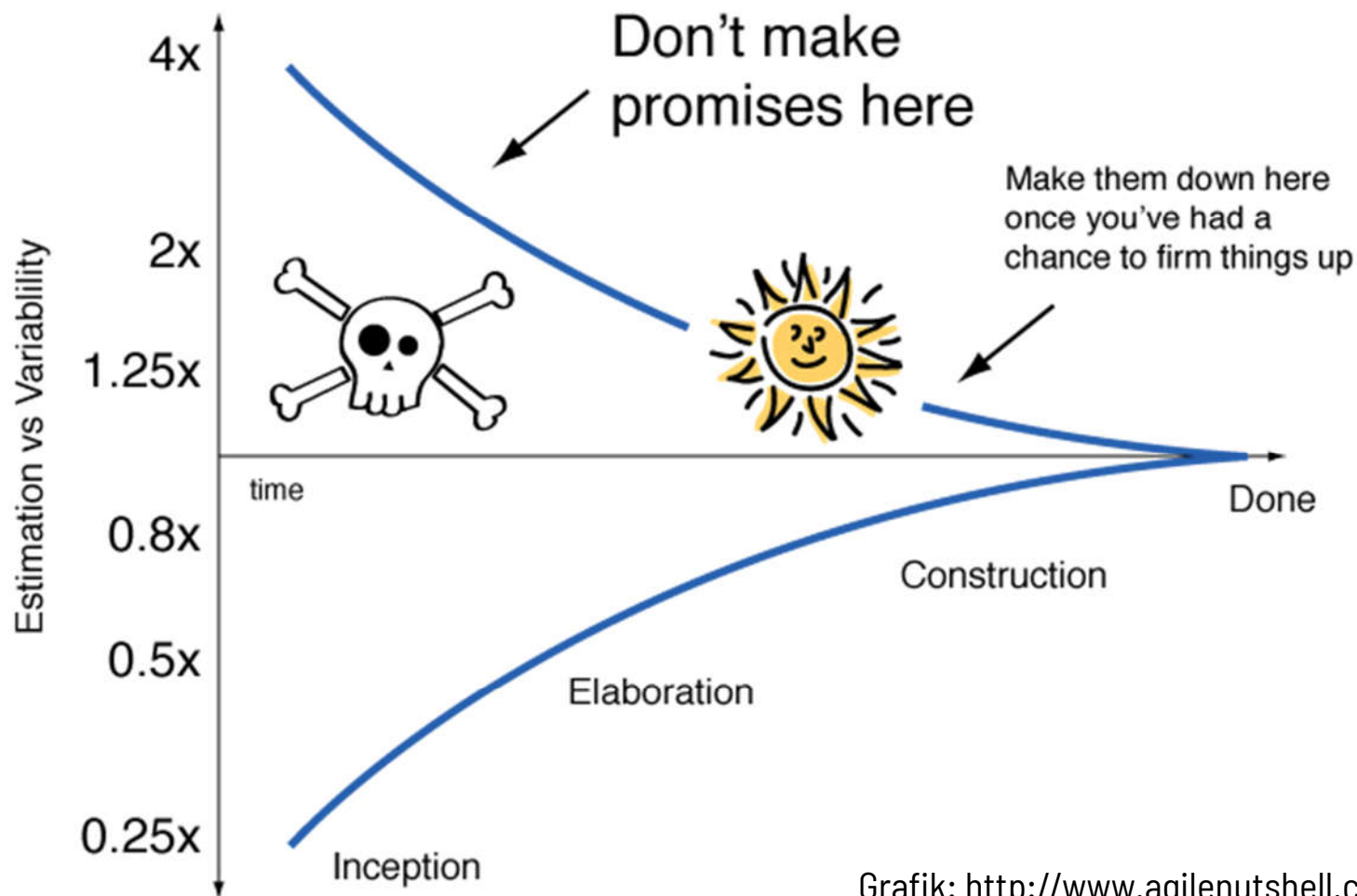
Besser Steuern und Lernen in Iterationen

Software Engineering-Prinzipien



Beispiel: Cone of Uncertainty

Software Engineering-Prinzipien



Iteratives
Vorgehen
unterstützt:
In unsicherem
Terrain mit kleineren
Schritten nach vorne

Grafik: http://www.agilenutshell.com/cone_of_uncertainty

Software Engineering-Prinzipien

Eine typische Nutzung

- Zu Beginn: Unklares und unvollständiges Verständnis der Anforderungen an Software zwischen Kunden und Auftragnehmern
- Vollständiges Verständnis braucht Zeit und ist schwierig, Missverständnisse sind teuer
- Inkrementell-iteratives Vorgehen unterstützt durch abgeschlossene Abschnitte
- Die Möglichkeit, Software später noch leicht und billig ändern zu können, unterstützt ebenfalls, etwa durch:
 - Lose Kopplungen zwischen Komponenten,
 - die gut gekapselt sind und
 - eine hohe Kohäsion aufweisen

